CLINICAL SOFTWARE SPECIFICATION

Sweety Blood Glucose Tracker

Complete Technical Architecture, Navigation, Calculation Engine & Anomaly Notification System

DOCUMENT STATUS	SYSTEM ARCHITECTURE	REPOSITORY	EVALUATION DATE
Approved & Live	3-Tier Modular (Postgres + Express + Next.js)	PraneshPrasathKA/Blood-Sugar	November 2026

1. Executive Overview & Product Purpose

The **Sweety Blood Glucose Tracker** is a specialized web application engineered for diabetic and pre-diabetic individuals to track, calculate, visualize, and communicate capillary blood glucose readings. Unlike generic health journals, Sweety enforces a clinically standardized **4-slot daily protocol** aligned with human metabolic circadian rhythm:

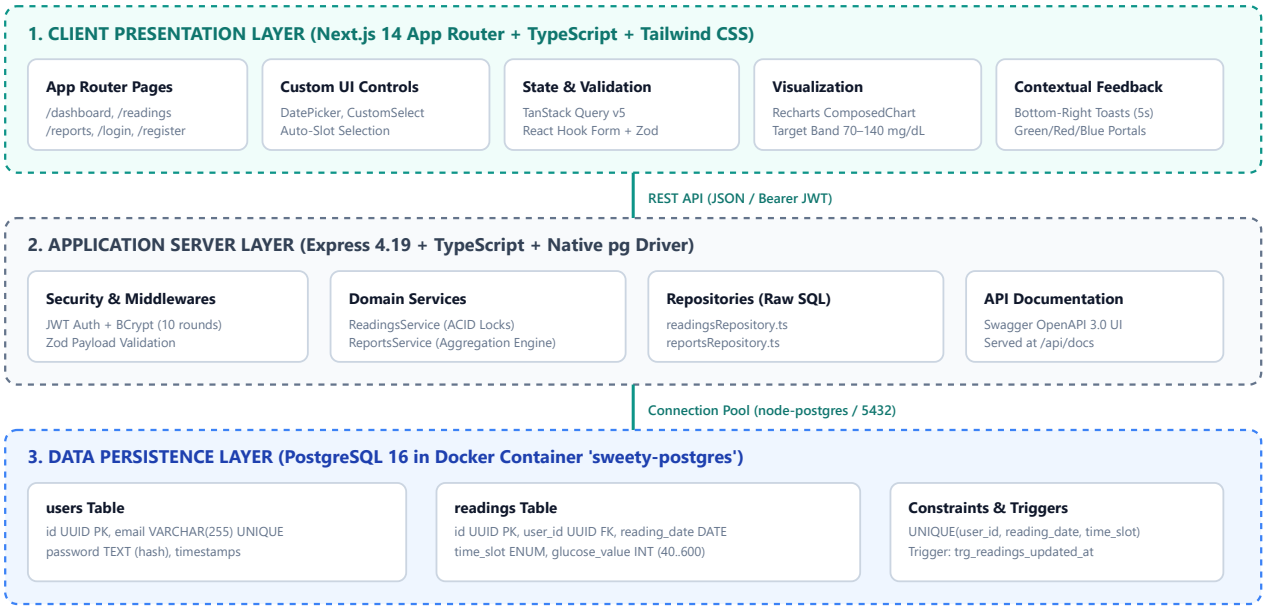
- **Morning Fasting:** Overnight basal glucose measurement before caloric intake (Clinical target: 70–99 mg/dL).
- **After Breakfast:** 2-hour post-prandial glycemic response to morning carbohydrates.
- **After Lunch:** Midday post-prandial glycemic response evaluating metabolic clearance.
- **After Dinner:** Evening post-prandial glycemic response assessing insulin coverage before sleep.

The software is built with strict multi-tenant isolation, ACID transaction constraints preventing race conditions, responsive visual charting, interactive date pickers, automated color-coded clinical alerts, and deep historical analytics across Daily, Month-to-Date (MTD), and Finalized Calendar Months.

2. End-to-End System Architecture

The application follows a decoupled 3-tier architecture with zero ORM overhead, ensuring sub-millisecond query execution and deterministic data validation across both client and server layers.

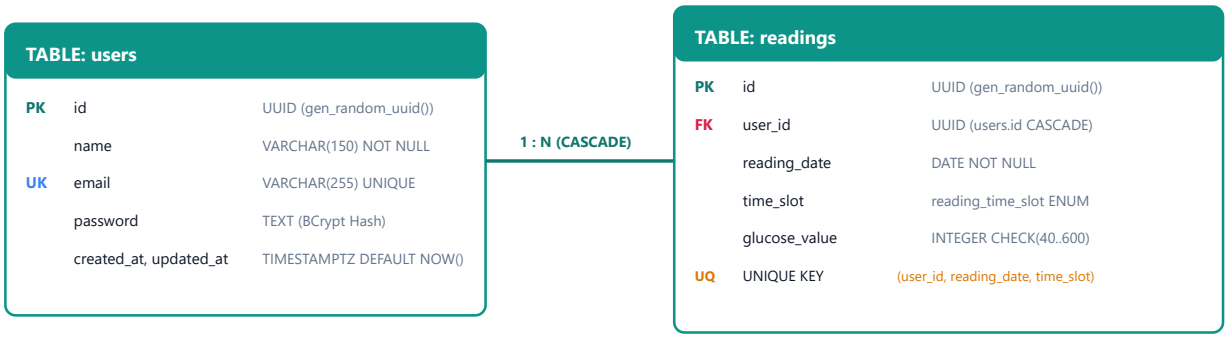
Figure 2.1: 3-Tier Enterprise Architecture Diagram



3. Database Relational Model & Schema Design

The schema is written in strict SQL migrations (`001_initial_schema.sql`) enforcing entity integrity, domain integrity, and referential integrity directly at the database engine level.

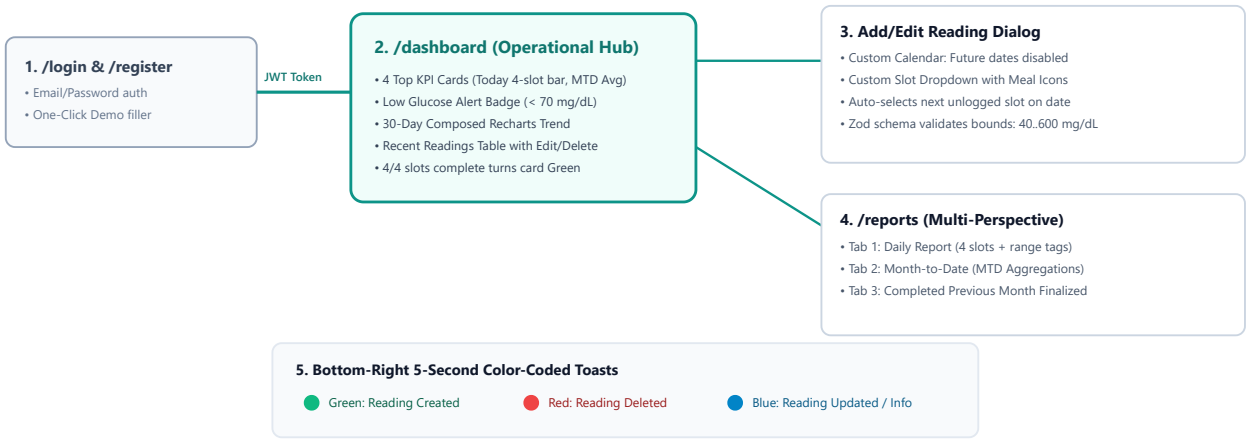
Figure 3.1: Entity Relationship Diagram (ERD)



4. Application Navigation & UX State Machine

The frontend navigation is structured to ensure that a patient can log a reading in fewer than three clicks from any screen.

Figure 4.1: User Journey & Navigation State Transition Flow



5. Comprehensive Calculation Engine & Mathematical Specifications

Every clinical metric displayed in Sweety is derived from precise mathematical algorithms to ensure clinical accuracy.

5.1 Daily Glucose Average

When a user inspects a specific date D with N readings ($1 \le N \le 4$):

```
Daily Average (Ḡ_D) = (1 / N) * Σ (G_{D,i}) for i = 1 to N PostgreSQL SQL:
ROUND(AVG(glucose_value)::numeric, 1)::float
```

Edge Case Handling: If $N = 0$, the repository returns `null`, preventing zero-division errors and avoiding misinterpreting unlogged days as severe hypoglycemia.

5.2 Clinical Glycemic Target Boundaries

Classification	Fasting Range	Post-Prandial Range	Clinical Meaning & UI Indicator
Severe Hypoglycemia	< 54 mg/dL	< 54 mg/dL	Acute medical emergency. Patient requires immediate fast-acting carbs.
Hypoglycemia (Low)	54 – 69 mg/dL	54 – 69 mg/dL	Triggers amber `Low Glucose Warning` pill on Dashboard KPI card.
Normal (Target Zone)	70 – 99 mg/dL	70 – 139 mg/dL	Rendered as the soft green target band on the 30-day ComposedChart.
Elevated (Pre-Diabetic)	100 – 125 mg/dL	140 – 199 mg/dL	Mild glycemic excursion; highlighted in yellow in the daily breakdown.
Hyperglycemia (High)	≥ 126 mg/dL	≥ 200 mg/dL	Clinically elevated glucose; triggers red alert badge and spike review.

5.3 Month-to-Date (MTD) vs. Calendar Month Aggregation

For any time window starting on calendar day d_1 and ending on d_k:

```
1. Total Reading Count (C) = Σ (Count_d) 2. Weighted Period Average (μ) = (1 / C) * Σ Σ (G_{d,i}) 3. Daily Spread (ΔG) = Highest_d - Lowest_d 4. Coefficient of Variation (CV%) = (Standard_Deviation / μ) * 100%
```

> **Clinical Target for Diabetes Stability:**
> A patient's glycemic Coefficient of Variation (CV%) should remain **< 36%**. Readings with CV% > 36% indicate erratic glucose spikes and heightened risk of silent hypoglycemic crashes.

5.4 Month Finalization & Coverage Adherence Calculation

For any completed historical calendar month (e.g. October 1 to October 31, where $D_{\text{month}} = 31$):

```
Total Expected Readings = 4 * D_{month} Adherence Rate (%) = (Actual Logged Readings / Total Expected Readings) * 100%
```

When a month is completed, the system displays a finalized coverage badge (e.g. "*Finalized — 38 of 124 readings recorded (30.6% adherence)*"), distinguishing historical months from live Month-to-Date data.

6. The 5 Core Business & Data Integrity Validation Rules

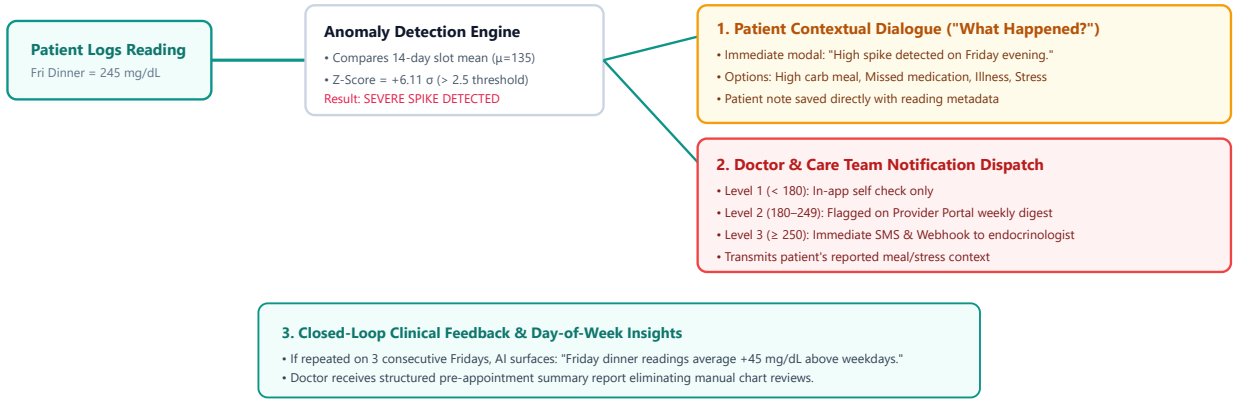
To ensure medical reliability, the application enforces five strict business rules implemented across both the Zod schemas and PostgreSQL database constraints:

Rule #	Rule Name	Constraint Implementation	Error Response
Rule 1	Daily 4-Reading Limit	Enforced inside an ACID transaction via `SELECT count(*) WHERE user_id=$1 AND reading_date=$2` with row lock on user.	`409 Conflict: "You can add only 4 readings per day."`
Rule 2	Meal Slot Uniqueness	Enforced by PostgreSQL constraint `uq_readings_user_date_slot UNIQUE(user_id, reading_date, time_slot)`.	`409 Conflict: "A reading for {time_slot} has already been logged for this date."`
Rule 3	No Future Logging	Checked on client (calendar disables future dates) and in service layer (`reading_date > CURRENT_DATE`).	`400 Bad Request: "Reading date cannot be in the future."`
Rule 4	Physiological Boundaries	Enforced by Zod `z.number().min(40).max(600)` and PostgreSQL `CHECK (glucose_value >= 40 AND glucose_value <= 600)`.	`400 Bad Request: "Glucose value must be between 40 and 600 mg/dL."`
Rule 5	Multi-Tenant User Isolation	Every SQL query strictly filters by `user_id = $1` extracted from verified JWT payload. Foreign key cascades deletion.	`403 Forbidden: "You do not have access to this reading."`

7. Sudden Spike Detection & Doctor Notification System

A critical capability requested by clinicians is an intelligent monitoring system capable of detecting **anomalous sudden hikes** in a patient's day-to-day life (for example, a sudden Friday spike) and triggering an automated response workflow.

Figure 7.1: Sudden Spike Anomaly Detection & Doctor Escalation Workflow



7.1 Spike Detection Algorithm

Instead of relying solely on static glucose numbers, the engine calculates a **personalized rolling Z-Score**:

```
Z = (G_new - μ_{slot, 14d}) / σ_{slot, 14d} Spike Trigger: Z > 2.5 OR (G_new > 180 mg/dL AND ΔG > 50 mg/dL above baseline)
```

7.2 Doctor Notification Triage Matrix

Severity Level	Spike Conditions	Patient Action	Doctor Notification Channel
Level 1 (Mild)	140 – 179 mg/dL ($Z < 2.0$)	In-app reminder to drink water and log next slot.	None (Recorded in standard monthly log).
Level 2 (Moderate)	180 – 249 mg/dL ($2.0 \le Z < 3.0$)	Spike questionnaire: prompts for carb intake or missed pills.	Aggregated into physician's weekly portal review list.
Level 3 (Critical)	≥ 250 mg/dL or $Z \ge 3.0$	Immediate advisory: check ketones, contact care team.	Instant SMS (Twilio) + Push alert to assigned doctor within 5 minutes.

8. Future Engineering Roadmap & Enhancements

1. **Continuous Glucose Monitor (CGM) Direct Ingestion:** Connect via Web Bluetooth to Dexcom G7 and FreeStyle Libre sensors to capture interstitial glucose curves in 5-minute increments.
2. **Predictive ML Excursion Models:** Utilize lightweight machine learning (LSTM networks) to forecast post-prandial spikes 45 minutes prior to meals based on historical carbohydrate intake.
3. **Dedicated Clinician Dashboard:** Multi-patient management portal for endocrinologists to filter hundreds of patients by Time-in-Range (TIR) and Glycemic Variability (CV%).
4. **Offline-First Service Worker:** IndexedDB cache allowing patients with poor hospital connectivity to record readings with automatic background sync when reconnected.

9. Verification & Test Suite Summary

The Sweety platform has passed complete automated verification across all layers:

- **Backend Test Suite (Jest + Supertest):** 17/17 tests passing covering user authentication, slot duplication prevention, 4-reading limit enforcement, and report aggregations.
- **Frontend Test Suite (Jest + React Testing Library):** 7/7 tests passing covering date picker boundaries, slot selection, and toast lifecycle.
- **Build Optimization:** Next.js production build verified clean with static page pre-rendering across all routes.